Information Retrieval and Web Analytics (IRWA) 2026

Part 1
Text Processing and Exploratory Data Analysis

Group members

Name	Student ID
Mauro Paniagua Suetta	u232628
Martí Pascual Barluenga	u232621
Martí Pérez Rodríguez	u232620

GitHub repository

`https://github.com/Martiperez314/IRWA-2026-project`

Tag: `IRWA-2026-part-1`

How to run everything (environment, data files and notebook) is explained in `project_progress/part_1/README.md`.

20 October 2026

1 Introduction and dataset

The corpus is a fashion e-commerce catalogue (`fashion_products_dataset.json`) with 28,080 products taken from Flipkart. In this part we clean and prepare the documents so they can be indexed (text pre-processing and how we handle each field), and we explore the data to understand what we are working with. All the reusable functions are in `myapp/search/preprocessing.py`, so queries can go through exactly the same processing as the documents. The results and figures in this report come from the notebook `project_progress/part_1/part_1.ipynb`.

Fact	**Value**
Products	28,080 (all `pid` unique)
Fields per product	17
Empty `description`	11,150 (39.7%)
Missing `average_rating`	2,261 (8.1%)
Missing `brand`	2,009 (7.2%)
Missing `seller`	1,643 (5.9%)
Missing `discount` / `actual_price`	855 (3.0%) / 777 (2.8%)
Missing `selling_price`	2

Table 1: Basic facts about the corpus (empty strings and empty lists count as missing).

Two things about the raw data affect most of our decisions:

- Prices, discount and rating come as *text* (`"2,999"`, `"69% off"`, `"3.9"`), so they need to be parsed before we can compare or sort them.
- `product_details` is a list of small dictionaries with one key each (for example `{"Fabric": "Cotton Blend"}` followed by `{"Pattern": "Solid"}`), so we flatten it before using it as text or as attributes.

2 Text pre-processing (point 1.1)

The whole pipeline is the function `preprocess_text`. The statement lists the steps in a different order, but they need to *run* in this one, because we can only remove stop words once the text is split into tokens:

normalize → tokenize → remove punctuation → remove stop words → stem

Step	**Function**	**Tool / setting**	**Example**
Tokenization	`tokenize`	NLTK `word_tokenize`	text → list of tokens
Removing punctuation	`tokenize`	keep only letters and digits in each token	`100%` → `100`; `, . &` removed
Removing stop words	`remove_stopwords`	NLTK English list (198 words) + 1-letter tokens	`from`, `it`, `a`, `for` removed
Stemming	`stem`	Snowball (Porter2)	`sleeves` → `sleev`, `jeans` → `jean`
Anything else (bonus)	`normalize_text`	regular expressions, see list below	`look.Designed` → `look. designed`

Table 2: Pre-processing steps asked in the statement and where they are implemented.

Tokenization and punctuation. `word_tokenize` deals with contractions and punctuation better than just splitting on spaces. After that, we remove any character that is not a lowercase letter or a digit, and drop the tokens that end up empty (pure punctuation like `%`, `.`, `,` or `&`).

Stop words. We use the NLTK English list. We also drop single-letter tokens, which are usually leftovers like the `s` in `men's`, but we keep single digits.

Bonus: `normalize_text`. While looking at the descriptions we found several problems that the four basic steps don't fix, so we clean the raw text first:

1. **HTML entities, tags and URLs removed** (`&` → `&`): some descriptions have escaped HTML that would otherwise create tokens like `amp`.
2. **Split on `/` and literal `\n`** (`Calf/Crew` → `Calf Crew`): the tokenizer keeps `calf/crew` as one token, so neither word could be found.
3. **Unicode to ASCII** (curly quotes and accents become plain characters): the same word appeared with different quote characters (for example around `OKANE`), which created duplicate terms.
4. **Split words glued by punctuation** (`look.Designed` → `look. Designed`): about 1,200 descriptions are missing the space after a period or comma. Without this step we would get nonsense tokens like `lookdesigned`.
5. **All spellings of t-shirt unified to `tshirt`** (`T-Shirt`, `t shirt`, `tshirts`): it is the most common product type (Figure 2) and it is written in many ways. Without this, `t shirt` would leave a lone `t` (dropped as a 1-letter token) and only `shirt` would be indexed.
6. **Lowercase**: upper or lower case doesn't change the meaning (`Cotton` = `cotton`).
7. **Split hyphenated words** (`full-sleeve` → `full sleeve`): people type `full sleeve` without the hyphen, like in query 1.

Example. The first product of the corpus and the two validation queries:

- Title "Solid Women Multicolor Track Pants" → [`solid`, `women`, `multicolor`, `track`, `pant`]
- Query 1 "women full sleeve sweatshirt cotton" → [`women`, `full`, `sleev`, `sweatshirt`, `cotton`]
- Query 2 "men slim jeans blue" → [`men`, `slim`, `jean`, `blue`]

Decisions.

- **We keep numbers.** In fashion they actually mean something: sizes, "pack of 2", "100% cotton". If we removed them, a query like "pack of 3 socks" would lose information.
- **Snowball instead of Porter or lemmatization.** Snowball (Porter2) is the improved version of Porter: it fixes some of its mistakes and is just as fast. A lemmatizer needs part-of-speech tags to work well (without them, WordNet treats every word as a noun and leaves `designed` unchanged) and is much slower on 28k products. For product search we prefer the extra matches that stemming gives; a few over-stemmed words (`friendly` → `friend`) don't hurt here.
- **Queries must use the same function.** The index stores stems like `sleev`; if a query is not processed the same way (`sleeves`), it won't match anything. That's why `preprocess_text` lives in `myapp/` and not inside the notebook.
- **Effect on the vocabulary** (title + description): 10,606 distinct raw words, then 6,676 after normalizing and tokenizing, and 5,044 terms after removing stop words and stemming. That is 52% fewer terms (see Section 7.3).

3 Fields returned for every result (point 1.2)

The 14 fields that every search result has to return are all present in the dataset (the notebook check finds none missing): pid, title, description, brand, category, sub_category, product_details, seller, out_of_stock, selling_price, discount, actual_price, average_rating and url. They are listed in the constant `OUTPUT_FIELDS`.

- **Original values stay untouched for display.** `build_processed_corpus` doesn't overwrite the text fields; it adds new columns (`title_tokens`, `description_tokens`, ..., `all_tokens`) that are only used for searching. The user sees "Solid Women Multicolor Track Pants", not `solid women multicolor track pant`. Only the numeric fields are converted in place (Section 5), because the parsed number is what we want to show and compare.
- **`pid` is the document ID.** It is unique (28,080 / 28,080) and it is the key used in `validation_labels.csv`, so we never change it.
- **Fields we don't use for retrieval:** `_id` (an internal ID from the crawler, we already have `pid`), `images` (thumbnail links with no searchable text) and `crawled_at` (when the page was crawled, which says nothing about relevance). They stay in the table but are not tokenized or indexed.

4 Category, sub-category, brand, product details and seller (point 1.3)

Option	Pros	Cons
1. Merge all fields into one text field	Simplest: one index, one vocabulary, one score. A query word matches wherever it appears (e.g. `cotton` in the details).	We lose *where* a word appeared: a word in the title counts the same as one lost in a long description, and long fields dominate.
2. Index each field separately	We can give each field its own weight (e.g. boost the title) and search a specific field (brand = Levi's).	Bigger index, and the ranking has to combine several scores. Short fields give unstable statistics.
3. Use some fields only as filters	Exact and cheap; a good fit for fields with very few values (`category`).	Words typed by the user ("cotton", "slim") would never match if those fields are not indexed as text too.

Table 3: Options for handling the category-like and semi-structured fields.

Data fact	Number
Distinct `category` values	4 (27,118 of 28,080 are "Clothing and Accessories")
Distinct `sub_category` values	24, some contain seller names
Distinct brands / sellers	324 / 534
Products whose brand is a cut-off seller name ("Lev", "SayItLo")	8,414
Distinct `product_details` keys	127
Most frequent keys	Fabric (25,959), Style Code (25,763), Pattern (25,692)

Table 4: Data we used to decide how to handle each field.

How useful is each field for telling products apart?

- `category` almost never helps: 96.6% of the products have the same value. As a term its IDF is close to zero, so merging it barely hurts, but it doesn't help either. It is more useful as a filter (e.g. Footwear).
- `sub_category` (Topwear, Bottomwear, Winter Wear...) is a bit more useful, but some values have seller names mixed in ("Uber Urban Clothing and Accessories").
- `brand` matters when someone searches for a brand, but it is often cut off (`ECKO Unl`, `REEB`, `Pu`). In 8,414 products the brand is just the start of the seller name, so a search for "puma" or "reebok" wouldn't find it.
- `product_details` has exactly the attributes people search for (fabric, colour, fit, sleeve, neck, *Ideal For*). Section 6 shows that `cotton` from query 1 never appears in the titles of the labelled products, only in the description and the details.
- `seller` is a shop name (RetailNet, SandSMarketing) that people rarely type.

Our choice: a mix of options 1 and 2. We keep one token column per field (`title_tokens`, `description_tokens`, `brand_tokens`, `category_tokens`, `sub_category_tokens`, `details_tokens`, `seller_tokens`) *and* one merged field, `all_tokens` = title + description + brand + category + sub_category + details (constant `MERGED_FIELDS`). The merged field makes it easy to build a simple index, and the separate fields let us give more weight to the title, which is where the validation labels show the most useful terms. `seller` is tokenized but not merged, because shop names would just add noise to text matching. We still keep its tokens because, when the brand is cut off, the seller often has the full name and can help complete it.

Which `product_details` keys we index. Only the values of 24 useful keys (`USEFUL_DETAIL_KEYS`: Fabric, Pattern, Sleeve, Type, Ideal For, Fit, Color, Neck Type, Occasion, Closure, Pack of, ...) are turned into text. We leave out keys like *Style Code* (codes such as `1005COMBO2` that nobody types and that only grow the vocabulary), *Fabric Care* (washing instructions, almost the same in thousands of products) and *Country of Origin* (where it was made, not what it is), plus Yes/No attributes like *Reversible*, whose values mean nothing as text.

5 Stock, prices, discount and rating (point 1.4)

We do **not index these fields as text**. `to_number` turns them into numbers (and `out_of_stock` stays a boolean), and we store them as attributes of each product.

Field	Raw	Stored as	Missing values
`selling_price`	`"921"`	921.0	2, left as NaN
`actual_price`	`"2,999"`	2999.0	777, set to `selling_price`
`discount`	`"69% off"`	69.0	855, set to 0
`average_rating`	`"3.9"`	3.9	2,261, left as NaN + `has_rating` flag
`out_of_stock`	`false`	boolean	none

Table 5: How the numeric fields are converted.

Why not as text. A token like `921` or `69` wouldn't help a text query. Nobody types an exact price, and when people mention a price they mean a range ("under 500"), which a word match can't express. These tokens would also add thousands of numbers to the vocabulary and could get mixed up with useful numbers in the text (sizes, packs). As numbers, they can be used for **filtering** (hide out-of-stock products, maximum price, minimum rating) and as **ranking signals** (rating, discount) together with the text score.

Assumptions.

- **No actual price means no discount.** If there is no original price, the product is sold at its selling price, so `actual_price` = `selling_price` and a missing `discount` becomes 0. This keeps the three fields consistent.
- **We don't fill missing ratings with the average.** Using the mean (3.63) would make unrated products look exactly "average" and mix them with products that really were rated, and it would also distort the rating distribution. We keep NaN plus the `has_rating` flag, so we can decide later how to treat unrated products.
- **The 2 products with no selling price keep NaN:** we can't make up a price; they simply won't pass a price filter.

6 Hints from the statement

Hint 1: `validation_labels.csv`

The file has 40 labelled products: query 1 has 13 relevant and 7 not relevant, and query 2 has 10 and 10. All 40 `pid`s are in the corpus. Table 6 shows in how many labelled products each (processed) query term appears, per field.

Query term	**Title**	**Description**	**product_details**
	(relevant / not relevant)		
`women` (q1)	13 / 7	6 / 2	0 / 4
`full` (q1)	13 / 3	6 / 4	13 / 3
`sleev` (q1)	13 / 3	5 / 5	13 / 7
`sweatshirt` (q1)	13 / 3	9 / 4	4 / 0
`cotton` (q1)	0 / 0	12 / 7	7 / 6
`men` (q2)	7 / 2	0 / 0	7 / 2
`slim` (q2)	5 / 7	5 / 4	0 / 0
`jean` (q2)	10 / 10	6 / 7	8 / 6
`blue` (q2)	10 / 10	3 / 5	10 / 10

Table 6: Labelled products containing each query term, per field. `brand` and `sub_category` contain none of the terms.

- `cotton` never appears in the titles, only in the description and in `product_details`. If we only indexed titles, one of the five query words would never match. This is clear evidence that the details are worth indexing.
- `sweatshirt`, `full` and `sleev` in the title separate the two groups well (13 of 13 relevant vs. 3 of 7 not relevant): most non-relevant products for query 1 are t-shirts. So the title deserves more weight than other fields.
- Some labels can't be explained by the text. For query 2, `jean` and `blue` are in every title, `slim` appears in more non-relevant titles than relevant ones, and some women's jeans are labelled relevant for "men slim jeans blue". Products with the exact same title ("Slim Women Blue Jeans") even get different labels. So a search based only on words can't get these labels perfectly right, and with just 20 labelled products per query, any evaluation with them will be noisy.

Hint 2: NLTK

From NLTK we use the English `stopwords` list, the `word_tokenize` tokenizer (plus `sent_tokenize` in the EDA) and the `SnowballStemmer`.

Hint 3: keep `pid`

`pid` is unique (28,080 / 28,080) and we keep it unchanged as the document ID. The processed corpus (28,080 rows, 28 columns) is saved to `data/processed_corpus.pkl` so it can be reused without running the whole pipeline again.

7 Exploratory Data Analysis

For each analysis we say why we did it and what it tells us about building the search engine.

7.1 Word counting distribution

Titles are short and regular: 6.46 words on average. Only 60.3% of the products have a description; when they do, it has 49.37 words on average, which become 31.13 terms after pre-processing (37% of the words are stop words or punctuation). We checked this because document length matters for ranking: in a title a word appears once or twice at most, so its weight depends mostly on how rare it is (IDF), while descriptions are long enough to need length normalization. The 39.7% of products without a description can only be found through their title and details, one more reason to index `product_details`.

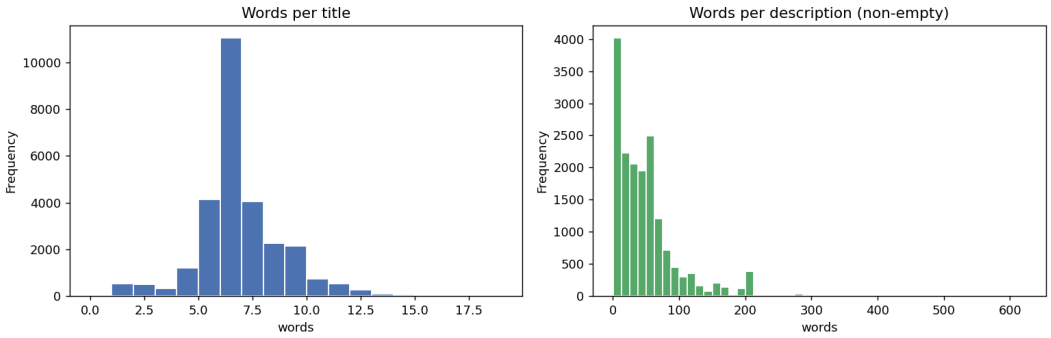


Figure 1: Words per title (left) and per non-empty description (right).

7.2 Average sentence length

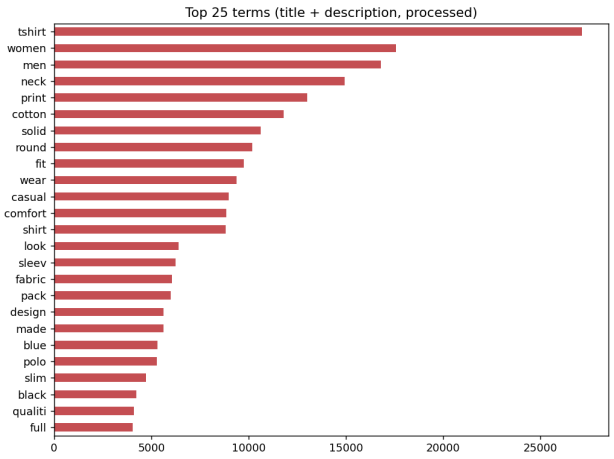
Descriptions have 3.24 sentences of 15.44 words on average (NLTK `sent_tokenize` on the normalized text, which already fixes the missing spaces after periods). They are short marketing texts, so a whole description fits easily in a result snippet.

7.3 Vocabulary size

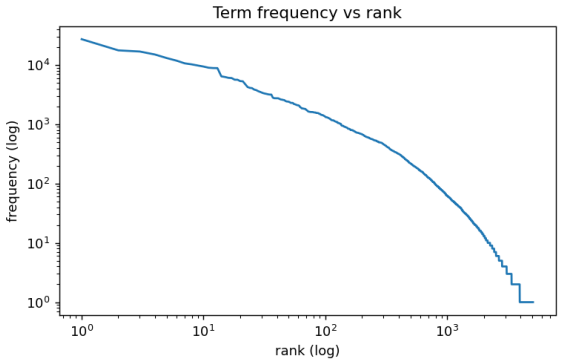
Stage (title + description)	Distinct terms
Raw words (split on spaces, lowercase)	10,606
Normalized + tokenized	6,676
+ stop words removed + stemmed	5,044
`all_tokens` (merged field, adds brand, categories and details)	5,790

Table 7: Vocabulary size after each step of the pipeline.

This shows what each step does: normalizing removes most of the noise (glued words, punctuation stuck to words) and stemming merges variants of the same word. The vocabulary is small because the corpus is very repetitive (see Section 7.9). The frequency vs. rank plot follows Zipf's law, and 1,123 terms (22%) appear only once (rare words, typos and codes); those will get a very high IDF.



(a) Top 25 processed terms.



(b) Term frequency vs. rank (log scale).

Figure 2: Most frequent terms and Zipf's law in titles and descriptions.

7.4 Ranking of products by rating, price and discount

We looked at these because they are the fields used for filtering and ranking. Selling prices are skewed to the right (median 545, mean 706, from 99 to 7,999), discounts are big (median 52%, up to 87%) and the mean rating is 3.63 (2,261 products have no rating). Rating and price are not correlated (−0.01), so a higher price doesn't mean better quality, while discount goes slightly down as the selling price goes up (−0.30). Many products share the top rating of 5.0, so sorting by rating alone gives lots of ties; we broke them by discount, and in general rating should be combined with the text score rather than used alone.

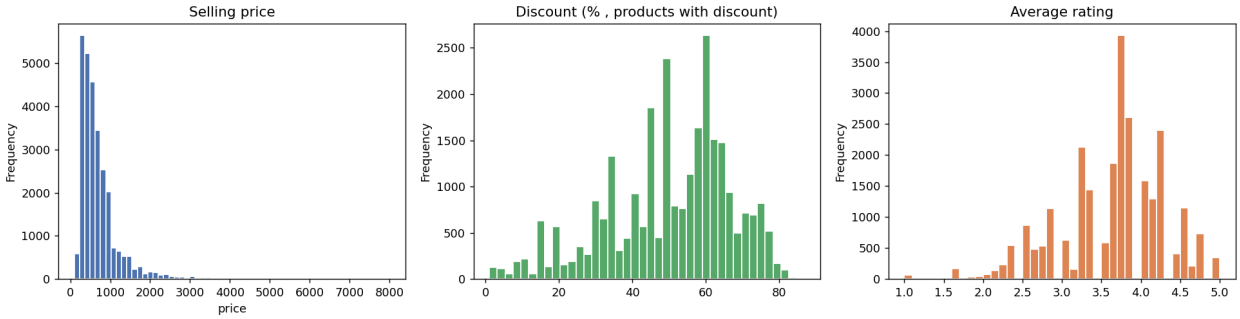


Figure 3: Distribution of selling price, discount (products with a discount) and average rating.

7.5 Top sellers and brands

There are 324 brands and 534 sellers. The most common brand names are clearly cut off (`ECKO Unl`, `Free Authori`, `REEB`, `Pu`, `True Bl`), which matches the 8,414 truncated brands from Section 4. The biggest seller, RetailNet, has more than 1,400 products. Among brands with at least 50 rated products (so a brand with a single 5-star product doesn't win), the best rated are SORA (4.61), PixF (4.46) and Cots (4.44). Searching by brand won't work well unless the cut-off names are completed (for example, using the seller).

7.6 Out-of-stock distribution

Only 5.9% of the products are out of stock, but it depends a lot on the sub-category: about 17% in Clothing Accessories and 14% in Tracksuits, but 0% in Fabrics. Since there are so few, hiding or pushing down out-of-stock products barely reduces the results and makes the user experience better.

Ranking	Title	Brand	Price	Disc.	Rating
Top rating	Men Solid Cotton Blend Pathani Kurta (Blue)	Purple Sta	449	82%	5.0
	Printed Men Boxer (Pack of 1)	SayItLo	249	78%	5.0
	Printed Women Round Neck Multicolor T-Shirt	yellowvib	275	77%	5.0
Top discount	Printed Women Round Neck White T-Shirt	Jack Roy	128	87%	3.0
	Printed Women Round Neck White T-Shirt	Jack Roy	136	86%	4.7
	Printed Women Round Neck Grey T-Shirt	Jack Roy	136	86%	4.1
Most expensive	Solid Women Track Suit	REEB	7,999	20%	2.7
	Checkered Single Breasted Party Women ...	True Bl	7,998	0%	2.6
	Full Sleeve Solid Women Casual Jacket	Pu	7,799	40%	3.7
Cheapest	Men Printed Bandana	T10 Spor	99	50%	2.4
	Women Mid-Calf/Crew (Pack of 2)	Welwe	99	50%	3.8
	Women Color Block Ankle Length (Pack of 3)	your shopping sto	118	76%	3.9

Table 8: Top 3 products of each ranking (the notebook shows the top 10).

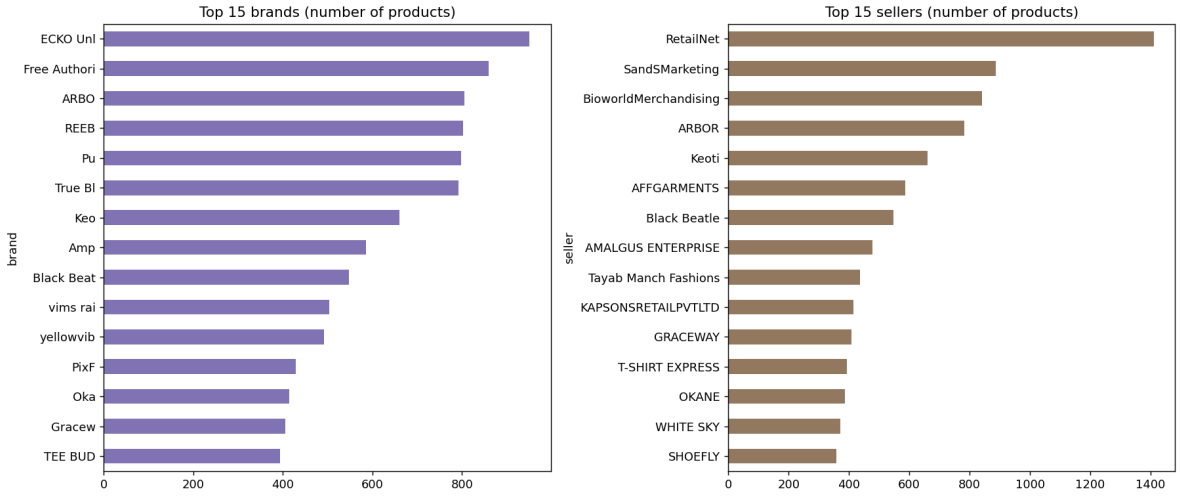


Figure 4: Top 15 brands and top 15 sellers by number of products.

7.7 Word clouds

We built the word clouds from the *processed* tokens, so they also work as a check of the pipeline (stems like `qualiti` show up). Titles are dominated by `women`, `men`, `tshirt`, `neck` and `solid`, and descriptions by `wear`, `cotton`, `fabric` and `comfort`. Words like `wear`, `look`, `made` or `comfort` are very frequent but say little about the product, so they behave like stop words for this domain. We don't remove them by hand because their low IDF already lowers their weight.

7.8 Entity recognition

We ran spaCy's small English model (`en_core_web_sm`) on a random sample of 2,000 descriptions (running it on all 17k takes too long) and got 5,578 entities, mostly ORG (1,668), PERSON (1,121), PERCENT and DATE. We used the *original* text, not the lowercased one, because the model relies

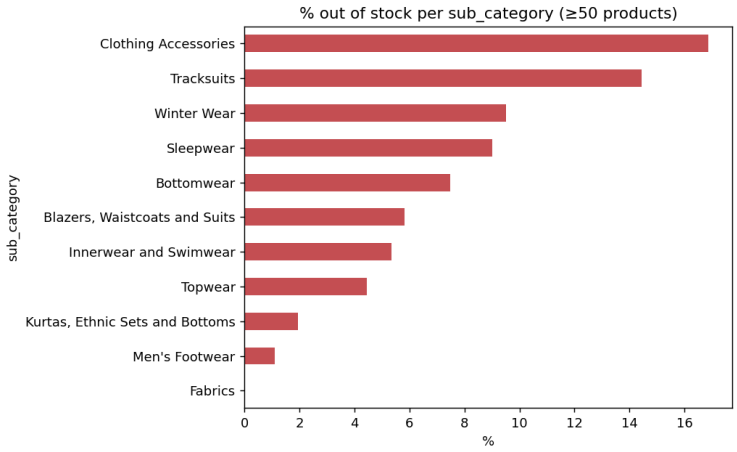


Figure 5: Percentage of out-of-stock products per sub-category (only sub-categories with at least 50 products).

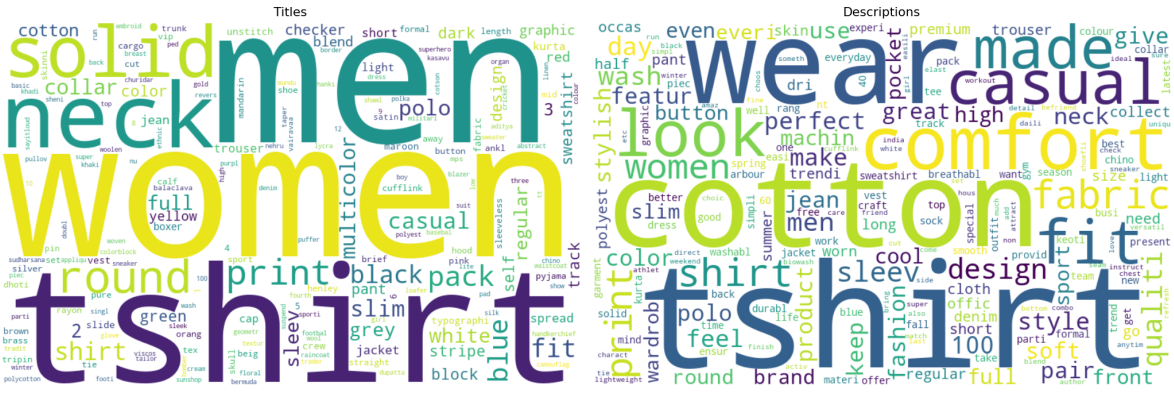


Figure 6: Word clouds of the processed titles (left) and descriptions (right).

a lot on capital letters to spot names. It finds some real brands (KEOTI, Levis, Lucky Bird) and places (India), but it makes many mistakes: "Cotton" is tagged as a place, "Casual Wear" and "Track Pant" as people, and "Printed" as an organization. The model was trained on news and web text, while product descriptions capitalize normal words for marketing, and fashion attributes (fabric, colour, fit) aren't among its entity types. So NER is not reliable for extracting attributes here, and we don't need it: `product_details` already gives them in a structured way.

7.9 Other findings

The categories are very unbalanced: 27,118 of 28,080 products are "Clothing and Accessories" (the rest are Footwear with 920, Bags, Wallets & Belts with 41 and Toys with 1). The corpus is also very repetitive: there are only 5,953 distinct titles and 5,683 distinct descriptions, and 42.1% of the products share title, description, brand and price with another product (for example, the same item in different sizes). For the search engine this means many products with exactly the same score, so result pages may need to remove duplicates, and evaluation has to keep near-duplicates in mind.

8 Conclusions and assumptions

The findings that matter most for building the search engine are: (i) 39.7% of the products have no description, so titles and `product_details` must be indexed, and `product_details` holds key attributes (fabric, colour, fit) that the queries use; (ii) the corpus is full of near-duplicate products (42.1%), which will cause ties in the ranking; (iii) brands are often cut off, so matching brands needs

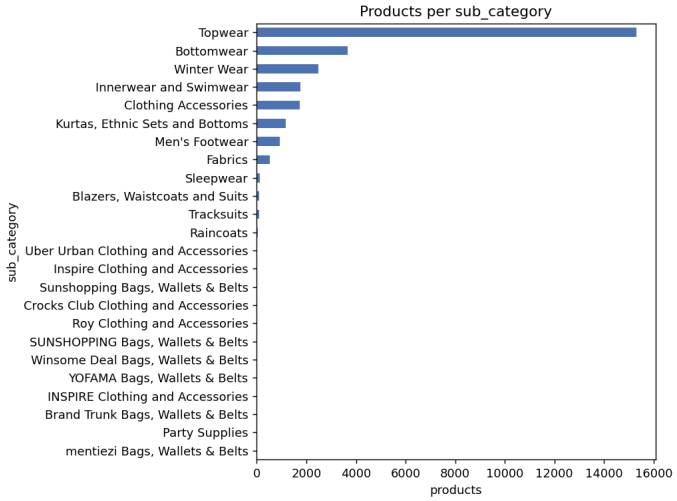


Figure 7: Number of products per sub-category.

the seller name; and (iv) the validation labels can't be fully explained by the text, which limits how well any word-based search can do on them.

Assumptions (summary).

- Empty strings and empty lists are treated as missing values.
- Missing `actual_price` becomes equal to `selling_price`; missing `discount` becomes 0 (no discount).
- Missing `average_rating` is not filled in (NaN + `has_rating`); the 2 missing selling prices stay NaN.
- Numbers in the text are kept (sizes, packs, percentages); prices and ratings are stored as numbers and not indexed as text.
- Only the values of the 24 keys in `USEFUL_DETAIL_KEYS` are indexed from `product_details`.
- The merged field `all_tokens` has title, description, brand, category, sub_category and details; `seller` has its own field but is not merged.
- `_id`, `images` and `crawled_at` are kept but not used for retrieval.
- Queries go through the same `preprocess_text` function as the documents.

9 AI usage

We used Claude (Anthropic) as a support tool: to review the code and suggest coding best practices, to help refactor it into reusable functions, to help us structure the report, and to format the documentation into LaTeX from a plain document we had written ourselves. We reviewed all its suggestions, ran the code ourselves and checked the final content.